Software Requirements Specification

FOR A QUIZ-BASED LEARNING PLATFORM

Members

Soham Thummar

Dushyant Dangar

Tannu Waghmare

Table of Contents

Table of Contents ..... **i**
Revision History ..... Error! Bookmark not defined.
1. Introduction **1**
1.1 Purpose 1
1.2 Document Conventions 1
1.3 Intended Audience and Reading Suggestions 2
1.4 Project Scope 2
1.5 References ..... **Error! Bookmark not defined.**
2. Overall Description **3**
2.1 Product Perspective 4
2.2 Product Features 4
2.3 User Classes and Characteristics 5
2.4 Operating Environment 8
2.5 Design and Implementation Constraints 8
2.6 User Documentation 8
2.7 Assumptions and Dependencies 9
3. System Features **10**
3.1 System Feature 1 ..... **Error! Bookmark not defined.**
3.2 System Feature 2 (and so on) 11
4. External Interface Requirements **12**
4.1 User Interfaces 16
4.2 Hardware Interfaces 16
4.3 Software Interfaces 16
4.4 Communications Interfaces 17
5. Other Nonfunctional Requirements **17**
5.1 Performance Requirements 17
5.2 Safety Requirements 17
5.3 Security Requirements ..... **Error! Bookmark not defined.**
5.4 Software Quality Attributes ..... **Error! Bookmark not defined.**
6. Other Requirements **18**
Appendix A: Glossary **19**
Appendix B: Analysis Models ..... Error! Bookmark not defined.
Appendix C: Issues List **20**

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) document is to provide a detailed description of the **Quiz Dash** system, including its functionality, system requirements, constraints, and user interactions. Quiz Dash is a web-based quiz management and learning platform designed to allow users to create, attempt, and manage quizzes in an interactive and user-friendly environment.

This document outlines the functional and non-functional requirements that guide the design and development of the Quiz Dash platform. It acts as a communication tool between the development team, project supervisors, and stakeholders to ensure that all participants clearly understand the expected system behaviour and objectives.

The SRS serves as a reference throughout the development lifecycle of the project. It helps developers understand the system features, assists testers in verifying system functionality, and ensures that the final product meets the defined requirements. By clearly documenting system specifications, this document helps reduce misunderstandings and ensures that the system is developed according to the intended goals.

This SRS is intended as a formal specification for the Quiz Dash architecture, logic-layer requirements, user workflows, scalability expectations, and security considerations for all future development and maintenance activities.

1.2 Document Conventions

This document follows standard Software Requirement Specification (SRS) conventions to clearly present the system requirements and functionalities. Headings and subheadings are organized in a hierarchical structure to improve readability and understanding.

Important terms related to system functionalities are written clearly to highlight key requirements. Functional requirements describe the actions the system must perform, while non-functional requirements specify performance, usability, and reliability aspects of the system.

Technical terms related to web development, quiz systems, and user interfaces are used where necessary to accurately describe system behaviour and features. The document is written in simple and clear language so that both technical and non-technical stakeholders can understand the system requirements.

1.3 Intended Audience and Reading Suggestions

This document is intended for all individuals involved in the development and evaluation of the **Quiz Dash** system. The primary audience includes project developers, instructors, project reviewers, and system testers.

Developers will use this document to understand the system requirements and implement the necessary functionalities during development. Project supervisors and evaluators can refer to this document to understand the system scope, objectives, and expected outcomes.

System testers can use the requirements described in this document to verify whether the developed system meets the specified requirements. Users who want to understand the system functionality can also refer to this document for a general overview of the Quiz Dash platform.

Readers who are interested in system features should refer to the functional requirements section, while those interested in technical constraints and system performance can refer to the non-functional requirements section.

1.4 Project Scope :

Quiz Dash is a web-based quiz management platform designed to provide an interactive and efficient environment for conducting and attempting quizzes online. The system allows users to register, log in, explore available quizzes, and test their knowledge through structured question-based assessments.

The platform enables administrators or quiz creators to create quizzes, add questions, manage quiz content, and monitor quiz performance. Users can attempt quizzes, view their scores, and track their performance over time. The system is designed with an intuitive user interface to ensure a smooth and engaging user experience.

Quiz Dash aims to simplify the process of quiz creation and participation by providing an organized dashboard where users can easily navigate through quizzes and view results. The platform focuses on improving learning engagement by offering quick feedback and performance insights.

The main goal of the Quiz Dash system is to provide a simple, scalable, and user-friendly platform for quiz-based learning and assessment that can be used by students, educators, or any organization that requires an online quiz system.

1.5 References.

The following resources were referred to during the preparation and design of the QuizDash system:

1. **Software Engineering – A Practitioner's Approach** by Roger S. Pressman, for standard Software Requirements Specification structure and documentation guidelines.
2. **Microsoft .NET Documentation**, used as a reference for backend development and system implementation.
3. **React.js Official Documentation**, used for understanding frontend development and user interface design principles.
4. **Figma Design Tool**, used to design and visualize the user interface of the QuizDash platform.
5. Various online resources and tutorials related to web development, quiz management systems, and user interface design.

2. Overall Description

2.1 Product Perspective

Quiz Dash is a standalone web-based application developed to provide an interactive platform for creating, managing, and attempting quizzes online. The system is designed to simplify quiz-based learning by providing an organized dashboard where users can easily access quizzes, track their performance, and view results.

The platform consists of a frontend interface developed using modern web technologies and a backend system responsible for handling user authentication, quiz management, and result processing. The system interacts with a database to store user information, quiz questions, answers, and quiz results.

Quiz Dash is designed to support multiple users simultaneously, allowing users to register, log in, attempt quizzes, and view their performance history. Administrators or quiz creators can manage quiz content, add or modify questions, and monitor quiz results through a centralized dashboard.

The system is designed to be scalable and user-friendly so that it can be easily extended in the future with additional features such as advanced analytics, leaderboard systems, or category-based quizzes.

2.2 Product Features

The Quiz Dash system provides several core functionalities that allow users to interact with quizzes and manage quiz-related activities efficiently. The major functions of the system are listed below:

- **User Registration and Login**

  The system allows new users to create an account by providing necessary details. Registered users can securely log in to access the platform and its features.

- **User Dashboard**

  After logging in, users can access a dashboard where they can view available quizzes, recent activities, and their performance statistics.

- **Quiz Categories**

  Quizzes are organized into categories such as NEET, JEE, NDA, SSC CGL, etc. Users can browse quizzes based on their preferred subject or exam category.

- **Quiz Playing System**

  Users can attempt quizzes and answer multiple-choice questions. Scores are calculated based on correct answers.

- **Practice Mode**

  Users can practice questions without competition. Helps users improve knowledge and prepare for quizzes.

- **Quiz Battle System**

  A real-time **1v1 battle system** where users compete with another randomly matched player. Both players answer the same questions, and the winner is determined by score and response time.

- **Solo Quiz Mode**

  Users can attempt quizzes individually without competing against other players.

- **Quiz Tournaments**

  Users can participate in tournaments organized by category. Participants compete with multiple users and rankings are generated based on scores.

- **Leaderboard**

  Displays rankings of users based on quiz performance and tournament scores. Encourage competition among participants.

- **Instructor Quiz Management**

  Instructors can create quizzes and add questions. They can manage quizzes they have created.

- **Admin Management System**

Administrators can manage users, questions, tournaments, and reports.

- **News & Updates**

  Displays announcements, updates, and information related to quizzes or tournaments.

- **Bug Reporting System**

  Users can report issues or errors encountered on the platform.

2.3 User Classes and Characteristics

The Quiz Dash system is designed to support different types of users who interact with the platform based on their roles and responsibilities. The main user classes are described below:

1. Administrator

The administrator has the highest level of access in the system. Administrators are responsible for managing quizzes and maintaining the overall system functionality. They can create, edit, or delete quizzes and questions. They may also monitor user activities and review quiz results. Administrators are expected to have basic technical knowledge and familiarity with web-based systems to manage quiz content effectively.

2. Registered Users (Students/Participants)

Registered users are individuals who sign up on the platform to attempt quizzes and test their knowledge. They can log in to their accounts, view available quizzes, attempt quizzes, and view their scores and performance history.

These users are expected to have basic computer and internet usage skills. The system interface is designed to be simple and user-friendly so that users with minimal technical knowledge can easily interact with the platform.

2.4 Operating Environment

The Quiz Dash system is a web-based application that operates in a standard web environment. Users can access the platform through modern web browsers such as Google Chrome, Mozilla Firefox, Microsoft Edge, or Safari.

The frontend of the system is developed using **React.js** to provide a responsive and interactive user interface. The backend is developed using **.NET**, which handles server-side logic, user authentication, quiz management, and communication with the database.

The system requires an active internet connection to access the platform and perform quiz-related activities. The application can run on different operating systems including Windows, macOS, and Linux, as long as a compatible web browser is available.

The system database stores user information, quiz questions, and results, ensuring efficient data management and retrieval during system operations

2.5 Design and Implementation Constraints

The development of the Quiz Dash system is subject to certain design and implementation constraints that must be considered during the development process.

The system is designed as a web-based application and must operate within standard web technologies and frameworks. The frontend of the system is implemented using **React.js**, while the

backend is developed using the **.NET framework**. These technologies are selected based on project requirements and development feasibility.

The system must run on commonly used web browsers and should follow standard web development practices to ensure compatibility and responsiveness across different devices.

Another constraint is related to database management, where the system must securely store user data, quiz questions, and quiz results. Proper validation and security mechanisms must be implemented to protect user information and maintain data integrity.

Additionally, the project is developed within the scope of an academic semester project, so the system features and complexity are limited to meet the given development time and resource constraints.

2.6 User Documentation

The Quiz Dash system will provide basic user guidance to help users understand how to use the platform effectively. The interface is designed to be simple and intuitive so that users can easily navigate through the system without requiring extensive instructions.

Users will be able to register, log in, select quizzes, attempt questions, and view results through clearly designed interface elements. Basic instructions and labels within the application will guide users during quiz attempts and other interactions.

In addition, a brief user guide or help section may be provided to explain key functionalities such as account creation, quiz participation, and viewing results. This documentation will assist new users in understanding the system features and using the platform efficiently.

2.7 Assumptions and Dependencies

The development and operation of the Quiz Dash system are based on certain assumptions and external dependencies.

It is assumed that users will have access to a stable internet connection and a device capable of running modern web browsers such as Google Chrome, Mozilla Firefox, or Microsoft Edge. The system also assumes that users possess basic computer and internet usage skills to interact with the platform.

The system depends on the proper functioning of the server environment and database system used to store user information, quiz questions, and results. Any interruption in the server or database services may affect the availability of the application.

The development of the system also depends on the availability of the chosen technologies and frameworks, such as **React.js** for frontend development and **.NET** for backend development. The project timeline and feature implementation are dependent on the academic schedule and available development resources.

3. System Features

3.1 User Registration

3.1.1 Description and Priority

This feature allows new users to create an account on the platform by providing their basic information. This is a high-priority feature because it enables users to access all system functionalities.

3.1.2 Functional Requirements

- *The system shall allow users to register by providing basic information such as name, email address, and password.*
- *The system shall validate user input before creating an account.*
- *The system shall ensure that each email address is unique.*
- *The system shall store the registered user details securely in the database.*
- *The system shall confirm successful registration to the user.*

3.2 User Login

3.2.1 Description and Priority

The system provides a login mechanism that allows registered users to securely access their accounts and use the quiz platform.

3.2.2 Functional Requirements

- *The system shall allow registered users to log in using their email and password.*
- *The system shall authenticate user credentials before granting access.*
- *The system shall display an error message if the login credentials are incorrect.*
- *The system shall redirect the user to the dashboard after successful login.*
- *The system shall allow users to log out of their account.*

3.3 Dashboard

3.3.1 Description and Priority

The dashboard provides users with an overview of available quizzes and their activity within the system. It acts as the central interface for navigating the platform.

3.3.2 Functional Requirements

- *The system shall display available quizzes on the user dashboard.*
- *The system shall allow users to select and start a quiz from the dashboard.*
- *The system shall display basic information about quizzes such as title or category.*

- *The system shall provide navigation options to access quizzes, results, and user profile.*

3.4 Quiz Attempt

3.4.1 **Description and Priority**

The system allows users to attempt quizzes by answering a set of multiple-choice questions.

3.4.2 **Functional Requirements**

- *The system shall display quiz questions with multiple answer options.*
- *The system shall allow users to select one answer for each question.*
- *The system shall allow users to navigate between questions during the quiz.*
- *The system shall submit the quiz once the user completes all questions or clicks submit.*
- *The system shall record user responses for evaluation.*

3.5 Result and Score Display

3.5.1 **Description and Priority**

After completing a quiz, the system evaluates the answers and displays the user′s score and performance..

3.5.2 **Functional Requirements**

- The system shall calculate the score based on correct answers.
- The system shall display the final quiz score to the user.
- The system shall indicate correct and incorrect answers.
- The system shall store quiz results for future reference.

3.6 Quiz Practice Mode

3.6.1 **Description and Priority**

This feature allows users to practice questions without participating in competitive quizzes.

3.6.2 Functional Requirements

- The system shall allow users to explore practice questions by category.
- The system shall display questions for practice purposes.
- The system shall allow users to view correct answers after attempting questions.
- The system shall help users improve knowledge through repeated practice.

3.7 Quiz Battle (1v1)

3.7.1 Description and Priority

This feature enables users to compete in a real-time one-on-one quiz battle with another randomly matched player.

3.7.2 Functional Requirements

- The system shall match users randomly with another available player.
- The system shall present the same set of questions to both players.
- The system shall calculate scores based on correct answers and response time.
- The system shall display the winner at the end of the battle.

3.8 Quiz Tournaments

3.8.1 Description and Priority

The system allows users to participate in quiz tournaments where multiple participants compete and rankings are generated.

3.8.2 Functional Requirements

- The system shall calculate scores based on correct answers and response time.

- The system shall allow users to join tournaments.
- The system shall calculate tournament scores based on quiz performance.
- The system shall calculate tournament scores based on quiz performance.

3.9 Leaderboard

3.9.1 Description and Priority

The leaderboard displays rankings of users based on their quiz scores and tournament performance.

3.9.2 Functional Requirements

- The system shall display rankings of top-performing users**.**
- The system shall update rankings based on quiz results.
- The system shall allow users to view their position on the leaderboard.

3.10 Quiz Creation (Instructor)

3.10.1 Description and Priority

This feature allows instructors to create and manage quizzes for students.

3.10.2 Functional Requirements

- The system shall allow instructors to create new quizzes.
- The system shall allow instructors to add questions and answer options.
- The system shall allow instructors to edit or delete quizzes.
- The system shall allow instructors to view quiz performance.

3.11 Admin Management

3.11.1 Description and Priority

The admin module allows administrators to manage users, questions, and tournaments within the system.

3.11.2 Functional Requirements

- The system shall allow admins to manage user accounts.
- The system shall allow admins to manage quiz questions.
- The system shall allow admins to create and manage tournaments.
- The system shall allow admins to monitor system activities.

3.12 Bug Reporting

3.12.1 Description and Priority

This feature allows users to report issues or bugs encountered while using the platform.

3.11.2 Functional Requirements

- The system shall allow users to submit bug reports.
- The system shall store reported issues in the system.
- The system shall allow administrators to review reported bugs.

4. External Interface Requirements

4.1 User Interfaces

The Quiz Dash system provides a clean and user-friendly graphical interface that allows users to easily interact with the platform. The interface is designed to ensure smooth navigation and an intuitive user experience.

The system includes several main user interface components such as the login page, registration page, user dashboard, quiz interface, and result display page. The login and registration pages allow users to securely access the platform. The dashboard serves as the main page where users can view available quizzes and access different system features.

During a quiz attempt, the system displays questions along with multiple answer options in a structured format. Users can select their answers and navigate through questions. After completing the quiz, the result page displays the user's score and performance details.

The interface is designed using modern web technologies to ensure responsiveness so that it can be accessed on different screen sizes and devices.

4.2 Hardware Interfaces

The Quiz Dash system does not require any specialized hardware components. Users can access the system using standard computing devices such as desktop computers, laptops, tablets, or smartphones.

The only hardware requirement is a device capable of running a modern web browser and connecting to the internet. Input devices such as a keyboard, mouse, or touchscreen are used to interact with the system interface.

4.3 Software Interfaces

The Quiz Dash system interacts with several software components during its operation. The frontend of the system is developed using **React.js**, which provides the user interface and manages user interactions.

The backend is implemented using the **.NET framework**, which handles server-side logic, authentication, quiz management, and communication with the database.

The system also interacts with a database management system that stores user information, quiz questions, and quiz results. The frontend and backend communicate through APIs to exchange data and ensure smooth system functionality.

4.4 Communications Interfaces

The Quiz Dash system uses standard internet communication protocols to exchange data between the client and the server. Communication between the frontend and backend components occurs through HTTP or HTTPS requests.

The system relies on a stable internet connection to allow users to access quizzes, submit answers, and retrieve results from the server. Secure communication protocols are used to ensure that user data and system information are transmitted safely.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

The Quiz Dash system should provide fast and efficient performance to ensure a smooth user experience. The system should load pages quickly and respond to user actions without noticeable delay.

The system should be capable of handling multiple users accessing the platform simultaneously without affecting performance. Quiz questions, results, and dashboard data should be retrieved and displayed efficiently from the database

5.2 Safety Requirements

The system must ensure the security and confidentiality of user data. User authentication mechanisms such as secure login and password protection must be implemented to prevent unauthorized access.

Sensitive user information such as passwords should be stored securely in the database using appropriate security practices. The system should also validate user inputs to prevent malicious activities such as unauthorized data access or manipulation.

5.3 Usability Requirements

The Quiz Dash platform should provide a simple and intuitive user interface that allows users to easily navigate through the system. The design should be clear and well-structured so that users can quickly understand how to register, log in, attempt quizzes, and view results.

The system should minimize complexity and provide clear instructions where necessary to ensure a positive user experience.

5.4 Reliability Requirements

The system should operate reliably and maintain consistent performance during normal usage. It should handle user requests accurately and ensure that quiz attempts and results are properly stored.

In case of unexpected errors, the system should display appropriate error messages and prevent data loss whenever possible.

5.5 Scalability Requirements

The system should be designed in a way that allows it to handle an increasing number of users and quizzes in the future. The architecture should support future enhancements such as additional quiz categories, advanced analytics, or leaderboard features without requiring major system redesign.

6. Other Non-Functional Requirements

6.1 Maintainability

The Quiz Dash system should be designed in a modular and well-structured manner so that future updates and maintenance can be performed easily. Proper coding practices, documentation, and clear separation between frontend and backend components should be maintained to simplify system modifications and debugging.

6.2 Portability

The Quiz Dash system should be accessible on different operating systems and devices without requiring significant changes. Since the platform is web-based, users should be able to access it through standard web browsers on Windows, macOS, or Linux systems.

6.3 Availability

The system should be available to users whenever they want to access quizzes or view their results. The server should ensure stable availability and minimize downtime to provide uninterrupted access to the platform.

Appendix A: Glossary

Term	*Meaning*
Quiz Dash	The web-based quiz management system described in this document.
User	A registered individual who logs in to the system to attempt quizzes and view results.
Administrator (Admin)	A user with special privileges who can create, edit, and manage quizzes.
Quiz	A set of questions created for users to test their knowledge.

Dashboard	The main interface where users can view quizzes, results, and system information.
Authentication	The process of verifying a user's identity during login.
Database	The system used to store user information, quiz questions, and results.
Frontend	The user interface part of the system developed using React.js.
Backend	The server-side part of the system developed using .NET that processes data and handles system logic.

Appendix C: Issues List

none